অধ্যায় ৫

Functions: Reusable Code, Parameters, Return Values এবং Scope

শেখার লক্ষ্য

এই অধ্যায় শেষে তুমি (১) Function Declaration, Expression ও Arrow Function লিখতে, (২) Parameter এবং Argument-এর পার্থক্য বুঝতে, (৩) Return Value দিয়ে ছোট ছোট Function যুক্ত করতে, (৪) Scope, Closure, Hoisting ও Recursion ব্যাখ্যা করতে, (৫) Pure Function ও Side Effect আলাদা করতে, (৬) একটি বাস্তব **Invoice Calculator** তৈরি ও Test করতে পারবে।

৫.১ Function কেন প্রয়োজন?

ধরো, একটি অনলাইন দোকানে তিন জায়গায় মোট দাম হিসাব করতে হচ্ছে। একই Calculation বারবার লিখলে একটি Bug ঠিক করার জন্য তিনটি জায়গায় পরিবর্তন করতে হবে। Function একটি নির্দিষ্ট কাজকে নাম দিয়ে পুনরায় ব্যবহারযোগ্য করে।

```javascript
function calculateLineTotal(price, quantity) {
  return price * quantity;
}
console.log(calculateLineTotal(120, 3)); // 360
console.log(calculateLineTotal(75, 2));  // 150
```

এখানে `calculateLineTotal` হলো Function Name; `price` ও `quantity` হলো Parameters; `120` ও `3` হলো Arguments; `return` Function-এর Result ফেরত দেয়।

মনে রাখবে: `console.log()` Output দেখায়, কিন্তু `return` Caller-কে Value দেয়। এগুলো একই কাজ করে না।

```javascript
function showTotal(a, b) {
  console.log(a + b);
}
const result = showTotal(4, 6); // Console: 10
console.log(result);            // undefined
```

৫.২ Function Declaration, Expression ও Arrow Function

```javascript
// Declaration
function add(a, b) { return a + b; }

// Expression
const subtract = function (a, b) { return a - b; };

// Arrow
const multiply = (a, b) => a * b;

console.log(add(2, 3));      // 5
console.log(subtract(8, 3)); // 5
console.log(multiply(4, 5)); // 20
```

Function Declaration সাধারণত Declaration-এর আগেও Call করা যায়, কারণ এটি Hoisted হয়। `const`-এ রাখা Function Expression বা Arrow Function তার Initialization-এর আগে ব্যবহার করা যায় না।

```javascript
sayHello(); // works
function sayHello() { console.log("Hello!"); }

// greet(); // ReferenceError: initialization-এর আগে access
const greet = () => console.log("Hi!");
```

Arrow Function-এর `this`: Arrow Function নিজের `this` Binding তৈরি করে না; চারপাশের Lexical Scope-এর `this` ব্যবহার করে। Object Method-এ `this` প্রয়োজন হলে সাধারণ Method Syntax অধিক উপযুক্ত।

```javascript
const account = {
  owner: "Rafi",
  describe() { return `Account: ${this.owner}`; }
};
console.log(account.describe());
```

৫.৩ Parameters, Default Values এবং Rest Parameters

```javascript
function greet(name = "Guest", prefix = "Welcome") {
  return `${prefix}, ${name}!`;
}
console.log(greet());
console.log(greet("Nila", "Hello"));
```

Default Parameter তখন কার্যকর হয় যখন Argument দেওয়া হয়নি অথবা Argument `undefined`। `null` পাঠালে Default বসে না।

```javascript
function sumAll(...numbers) {
  return numbers.reduce((sum, value) => sum + value, 0);
}
console.log(sumAll(10, 20, 30)); // 60
```

`...numbers` Rest Parameter: একাধিক Argument-কে একটি Array-তে সংগ্রহ করে। বিপরীতে Function Call-এর সময় `...array` ব্যবহার করলে সেটি Spread Syntax।

```javascript
const values = [5, 8, 12];
console.log(sumAll(...values)); // 25
```

৫.৪ Return, Early Return এবং Guard Clause

```javascript
function getShippingFee(total) {
  if (!Number.isFinite(total) || total < 0) {
    throw new TypeError("Invalid total");
  }
  if (total >= 2000) return 0;
  return 120;
}
```

Guard Clause অগ্রহণযোগ্য Input শুরুতেই Reject করে; Nested `if` কমিয়ে Code পড়া সহজ করে। `return` Execute হলে Function-এর পরের Statement আর চলে না।

৫.৫ Scope: Global, Function ও Block

```javascript
const shopName = "JS Store"; // module/global-level binding
function createReceipt() {
  const receiptId = "R-101"; // function scope
  if (true) {
    const message = `${shopName}: ${receiptId}`; // block scope
    console.log(message);
  }
  // console.log(message); // ReferenceError
}
createReceipt();
```

`let` এবং `const` Block Scoped; `var` Function Scoped। Browser Script-এর Top-Level `var`-এর Global Object-এর সঙ্গে সম্পর্ক এবং Module-এর Scope ভিন্ন হতে পারে। নতুন Code-এ সাধারণত `const` ও প্রয়োজনমতো `let` ব্যবহার করবে।

Shadowing

```javascript
const rate = 5;
function printRate() {
  const rate = 10;
  console.log(rate); // 10
}
printRate();
console.log(rate); // 5
```

Inner Binding একই নামের Outer Binding-কে ওই Scope-এ Shadow করে। অতিরিক্ত Shadowing বিভ্রান্তি সৃষ্টি করতে পারে।

৫.৬ Closure: Function কীভাবে State মনে রাখে?

Closure মানে Function তার তৈরি হওয়ার সময়কার Lexical Environment-এর Binding-এ Access ধরে রাখতে পারে—Outer Function Return করার পরও।

```javascript
function createCounter(start = 0) {
  let count = start;
  return function increment() {
    count += 1;
    return count;
  };
}
const counterA = createCounter();
console.log(counterA()); // 1
console.log(counterA()); // 2
const counterB = createCounter(10);
console.log(counterB()); // 11
```

প্রতিটি `createCounter()` Call আলাদা State তৈরি করে। Closure ব্যবহার হয় Event Handler, Configuration, Factory Function, Memoization এবং Encapsulation-এ। তবে বড় Data অপ্রয়োজনে Capture করলে Memory Retention হতে পারে।

৫.৭ Callback ও Higher-Order Function

Function JavaScript-এ First-Class Value; Variable-এ রাখা, Argument হিসেবে পাঠানো এবং Return করা যায়।

```javascript
function applyOperation(a, b, operation) {
  return operation(a, b);
}
const result = applyOperation(10, 5, (x, y) => x - y);
console.log(result); // 5
```

যে Function অন্য Function নেয় বা Return করে তাকে Higher-Order Function বলে। Array-এর `map`, `filter`, `reduce`-এ এটি প্রচুর ব্যবহৃত হয়।

```javascript
const prices = [100, 250, 500];
const discounted = prices.map(price => price * 0.9);
console.log(discounted); // [90, 225, 450]
```

৫.৮ Recursion: Function নিজেকেই Call করে

```javascript
function factorial(n) {
  if (!Number.isSafeInteger(n) || n < 0 || n > 170) {
    throw new RangeError("n must be an integer between 0 and 170");
  }
  if (n <= 1) return 1; // base case
  return n * factorial(n - 1); // recursive case
}
console.log(factorial(5)); // 120
```

Recursion-এ Base Case না থাকলে Stack Overflow হতে পারে। JavaScript Engine-এ Tail-Call Optimization সর্বত্র নির্ভরযোগ্য নয়। বড় Input-এর ক্ষেত্রে Loop কখনো অধিক নিরাপদ।

৫.৯ Pure Function বনাম Side Effect

```javascript
function addTax(amount, taxRate) {
  return amount + amount * taxRate;
}
```

একই Input-এ একই Output এবং বাইরে কোনো Observable State পরিবর্তন না হলে Function-কে Pure বলা হয়। DOM Update, Network Request, `console.log`, File Write, বাইরের Variable পরিবর্তন ইত্যাদি Side Effect। Side Effect নিষিদ্ধ নয়; বরং Calculation ও UI/IO Logic আলাদা রাখলে Testing সহজ হয়।

```javascript
const cart = [{ name: "Pen", price: 20 }];
function copyWithItem(items, newItem) {
  return [...items, newItem];
}
const updated = copyWithItem(cart, { name: "Book", price: 100 });
console.log(cart.length);    // 1
console.log(updated.length); // 2
```

৫.১০ Practical Project — Invoice Calculator

Project Goal

একটি ছোট দোকানের Invoice App বানাব, যেখানে Product Name, Unit Price এবং Quantity দিয়ে Item যোগ করা যাবে। Discount এবং উদাহরণমূলক Tax Rate বদলালে Grand Total আপডেট হবে। এখানে **Discount আগে, Tax পরে** প্রয়োগ করছি—এটি এই Project-এর নির্দিষ্ট Calculation Rule, কোনো বাস্তব Tax Law-এর দাবি নয়।

Project File Structure

```
source-code/chapter-05/
├── index.html
├── style.css
├── invoice.js
├── app.js
└── invoice.test.js
```

Design Decision: টাকা Floating-Point BDT হিসেবে না রেখে **Integer Paisa**-তে হিসাব করা হচ্ছে। উদাহরণ: `BDT 12.50` → `1250` Paisa। Rate রাখা হচ্ছে Basis Points-এ: `5% = 500 bps`, `10% = 1000 bps`। `Math.round` ব্যবহার করে Positive Amount-এর Half-Up Rounding দেখানো হয়েছে; Production Financial Software-এ প্রতিষ্ঠানের Rounding Policy অনুযায়ী সিদ্ধান্ত নিতে হবে।

Core Engine (`invoice.js`)

JAVASCRIPT

```javascript
function percentOf(amount, rateBasisPoints) {
  if (!Number.isSafeInteger(amount) || amount < 0) throw new TypeError("Invalid amount");
  if (!Number.isSafeInteger(rateBasisPoints) || rateBasisPoints < 0 || rateBasisPoints > 10000)
{
    throw new RangeError("Invalid rate");
  }
  const product = amount * rateBasisPoints;
  if (!Number.isSafeInteger(product)) throw new RangeError("Amount too large");
  return Math.round(product / 10000);
}

function calculateInvoice(lines, { discountBps = 0, taxBps = 0 } = {}) {
  let subtotal = 0;
  for (const line of lines) {
    subtotal += line.unitPricePaisa * line.quantity;
  }
  const discount = percentOf(subtotal, discountBps);
  const taxable = subtotal - discount;
  const tax = percentOf(taxable, taxBps);
  return { subtotal, discount, tax, total: taxable + tax };
}
```

উপরের Snippet মূল Algorithm বোঝানোর জন্য সরল করা। **Runnable `invoice.js`-এ পূর্ণ Input Validation, Safe Integer Check এবং Line Details আছে।** `app.js` DOM Update করে; `invoice.js` Pure Calculation Logic ধরে রাখে। এতে একই Logic Browser ও Node.js Test—দুই জায়গায় ব্যবহার করা যায়।

হিসাবের উদাহরণ

ধরা যাক একটি Item-এর দাম BDT 100.00, Quantity 1, Discount 10%, Example Tax 10%।

Step	Amount
Subtotal	BDT 100.00
Discount	BDT 10.00
Taxable amount	BDT 90.00
Example Tax	BDT 9.00
Grand Total	**BDT 99.00**

App চালানোর নিয়ম

1. `source-code/chapter-05/index.html` Browser-এ খোলো।
2. Product Name, Unit Price ও Quantity লিখে **Add Item** চাপো।
3. Discount ও Example Tax Rate পরিবর্তন করো।
4. Subtotal, Discount, Taxable Amount, Tax ও Grand Total দেখো।
5. **Clear** চাপলে Items মুছে যাবে।
6. Node.js থাকলে `node --test invoice.test.js` দিয়ে Automated Tests চালাও।

Code Architecture

```
User Input
   ↓
app.js → parseBdtToPaisa() → calculateInvoice()
   ↓                                  ↓
DOM Rendering ← Invoice Result Object
```

Security Note: User-provided Product Name `textContent` দিয়ে DOM-এ বসানো হয়েছে, `innerHTML` দিয়ে নয়। এটি সাধারণ HTML Injection এড়াতে সাহায্য করে।

৫.১১ Common Mistakes এবং Debugging

Mistake 1 — `return` ভুলে যাওয়া

```javascript
const wrong = (a, b) => { a + b; }; // undefined
const correct = (a, b) => { return a + b; };
```

Mistake 2 — Parameter-কে Type Validation ছাড়া ব্যবহার

```javascript
function add(a, b) { return a + b; }
console.log(add("5", 2)); // "52", 7 নয়
```

Mistake 3 — Default Parameter-এ `null` ধরে নেওয়া

```javascript
function display(name = "Guest") { return name; }
console.log(display(undefined)); // Guest
console.log(display(null));      // null
```

Mistake 4 — Closure-এর Shared State বুঝতে না পারা: একই Counter Function-এ একাধিক Call করলে State বাড়ে; নতুন Factory Call করলে আলাদা State তৈরি হয়।

Mistake 5 — Recursive Base Case ভুল: Recursion-এর আগে Exit Condition লিখে পরীক্ষা করো।

৫.১২ Practice Exercises

1. `square(number)` Function লিখো, যেটি Number-এর Square Return করে।
2. `isEven(n)` Function লিখো; Input Safe Integer না হলে Error দেবে।
3. `getFullName(first, last)` লিখো; `last` না থাকলে শুধু `first` Return করবে।
4. Rest Parameter দিয়ে `average(...numbers)` লিখো; Empty Input Reject করবে।
5. `createMultiplier(factor)` Closure লিখো, যা নতুন Function Return করবে।
6. `sumTo(n)` Recursion দিয়ে লিখো; `n` হবে 0–1000-এর Integer।
7. Invoice App-এ একটি নতুন `serviceFeePaisa` Option যুক্ত করো, যা Tax-এর পরে যোগ হবে।
8. Invoice App-এ Remove Item Button যোগ করো এবং Item Index Validation করো।

৫.১৩ Exercise Solutions

```javascript
// 1
const square = number => number * number;

// 2
function isEven(n) {
  if (!Number.isSafeInteger(n)) throw new TypeError("Integer required");
  return n % 2 === 0;
}

// 3
function getFullName(first, last = "") {
  return `${first} ${last}`.trim();
}

// 4
function average(...numbers) {
  if (numbers.length === 0 || !numbers.every(Number.isFinite)) {
    throw new TypeError("At least one finite number required");
  }
  return numbers.reduce((sum, n) => sum + n, 0) / numbers.length;
}

// 5
```

```
function createMultiplier(factor) {
  return value => value * factor;
}
const double = createMultiplier(2);
console.log(double(7)); // 14

// 6
function sumTo(n) {
  if (!Number.isSafeInteger(n) || n < 0 || n > 1000) throw new RangeError("Invalid n");
  if (n === 0) return 0;
  return n + sumTo(n - 1);
}
```

Solution 7 (Implementation Guide): `calculateInvoice()`-এর Option-এ `serviceFeePaisa = 0` যোগ করে Safe Integer Validation করবে। এরপর `total = taxable + tax + serviceFeePaisa` হিসাব করবে এবং Overflow Check করবে। Test-এ `serviceFeePaisa: 250` দিলে Grand Total 250 Paisa বাড়ছে কি না দেখবে।

Solution 8 (Implementation Guide): প্রতিটি List Item-এর সঙ্গে `Remove` Button তৈরি করে তার `data-index` বা Closure-bound Index ব্যবহার করো। Click Handler-এ Index `0 <= index < lines.length` যাচাই করে `splice()` দিয়ে Remove করো, তারপর `render()`। Button-এর Label ও Focus Accessibility বজায় রাখবে।

৫.১৪ MCQ — নিজেকে যাচাই করো

1. Function-এর Output Caller-কে কোন Keyword দেয়? (A) print (B) return (C) break (D) let
2. Function Declaration সাধারণত কোন বৈশিষ্ট্য রাখে? (A) Hoisting (B) Private this (C) No scope (D) Always async
3. Rest Parameter কোন Syntax ব্যবহার করে? (A) `??` (B) `...` (C) `=>` (D) `?.`
4. Arrow Function-এর `this` সাধারণত কোথা থেকে আসে? (A) New binding (B) Global always (C) Lexical scope (D) Argument
5. Closure কী সংরক্ষণ করতে পারে? (A) শুধু HTML (B) Lexical binding access (C) Browser History (D) CSS
6. `const f = x => { x * 2 }; f(3)` কী Return করে? (A) 6 (B) 3 (C) undefined (D) null
7. Base Case ছাড়া Recursion কী সমস্যা করতে পারে? (A) Stack overflow (B) CSS error (C) Syntax always (D) HTTP 404
8. Pure Function-এর মূল বৈশিষ্ট্য কী? (A) DOM বদলায় (B) Same input → same output, no side effects (C) Network call (D) Random output
9. `greet(undefined)`-এ Default Parameter কাজ করে? (A) হ্যাঁ (B) না (C) কেবল null-এ (D) কেবল Arrow Function-এ
10. Invoice Engine-এ Integer Paisa ব্যবহারের সুবিধা কী? (A) UI বড় হয় (B) Binary floating-point precision সমস্যা কমে (C) CSS লাগে না (D) Tax লাগে না

MCQ Answer Key ও Explanation

No.	Answer	Explanation
1	B	`return` Caller-কে Value দেয়।

No.	Answer	Explanation
2	A	Function Declaration Hoisted হয়।
3	B	`...args` Rest Parameters সংগ্রহ করে।
4	C	Arrow Function Lexical `this` ব্যবহার করে।
5	B	Closure Lexical Environment-এর Binding-এ Access ধরে রাখে।
6	C	Block Body-তে `return` নেই।
7	A	Exit Condition না থাকলে Call Stack শেষ হতে পারে।
8	B	Pure Function-এর Deterministic Output ও Side-Effect-Free Behaviour।
9	A	`undefined` Default Parameter Trigger করে।
10	B	Integer Minor Units ব্যবহার করলে Decimal Arithmetic-এর অনেক সমস্যা এড়ানো যায়।

৫.১৫ Interview Questions

Q: Parameter ও Argument-এর পার্থক্য কী? Parameter হলো Function Definition-এর Placeholder; Argument হলো Call-এর সময় পাঠানো Actual Value।

Q: Closure কি Outer Variable-এর Snapshot নেয়? সাধারণত না; Closure Lexical Binding-এ Access ধরে রাখে, ফলে Variable পরিবর্তিত হলে পরবর্তী Call-এ নতুন Value দেখা যেতে পারে।

Q: Arrow Function কি সবসময় Regular Function-এর বিকল্প? না। `this`, `arguments`, Constructor Usage এবং Generator Behaviour-এ গুরুত্বপূর্ণ পার্থক্য আছে।

Q: `return` ও `console.log` কেন আলাদা? `return` Computation Compose করতে সাহায্য করে; `console.log` মূলত Debug/Output Side Effect।

৫.১৬ Chapter Summary

- Function একটি Reusable Behaviour তৈরি করে।
- Parameter/Argument এবং Return Value Function Composition-এর ভিত্তি।
- Declaration, Expression ও Arrow Function-এর Scope/Hoisting/`this` পার্থক্য বুঝতে হবে।
- Closure, Callback ও Higher-Order Function আধুনিক JavaScript-এর গুরুত্বপূর্ণ Building Block।
- Pure Calculation Logic UI Code থেকে আলাদা রাখলে Testing সহজ হয়।
- Invoice Calculator Project-এ Validation, Rounding, Testability ও DOM Safety-এর ব্যবহার দেখা হয়েছে।

পরবর্তী অধ্যায়: Chapter 6 — Arrays: Data Collection, Array Methods এবং Practical Data Processing; Practical Project: Inventory Dashboard।